BiWrite User Guide

Version 0.2.11

October 2026

Source code https://github.com/zzccppp/biwrite
Writing skill research-builder https://github.com/qzkinhit/research-builder

Abstract

BiWrite is an editor for writing an academic paper in English and Chinese at once. You write on the left, the right pane shows the other language paragraph by paragraph, and only paragraphs you change are translated again. A LaTeX paper compiles into a PDF inside the app, unsaved edits included, and a click on a sentence in the PDF selects that sentence in the source. A writing assistant polishes, edits, answers questions, writes new paragraphs in the manner of a published paper, writes figures and repairs LaTeX errors, with the rules of the research-builder writing skill. Section 2 follows one paper from the template to submission, and the sections after it go through each feature step by step. In the screenshots, red boxes mark the buttons to use and the areas to look at, and the numbers in the text, such as **(1)**, refer to the numbers next to the boxes.

1 Features at a glance

Each feature name below links to its section.

The writing workflow
Ten steps from a new paper on a template, through writing, imitating a published paper, figures, the PDF and editing the Chinese, to the Chinese version.

Bilingual writing
English on the left, the Chinese paragraph by paragraph on the right, and only changed paragraphs are translated again. The swap button in the toolbar lets you edit the Chinese instead, with the English following and unchanged paragraphs kept word for word. Swapping early, paragraphs not translated yet are filled in on the left as their translations arrive.

A paper written in Chinese
A Chinese file opens as the Chinese side and is translated into English, and saving always writes the file's own language. A document taken for the wrong language is set right in one step.

LaTeX builds and PDF navigation
The PDF compiles inside the app. Unsaved edits compile in a cache folder, and the file on disk stays as last saved. A click on a sentence in the PDF selects it in the source, ready to polish. The paragraph at the cursor can be found in the PDF, and a build error opens its line.

Chinese PDF
Once the English is translated, the Chinese PDF compiles in one step, and the Chinese PDF and a stand-alone Chinese .tex can be exported.

Your existing Chinese version
`paper_zh.tex` or `sections_zh/` pair with the English paragraph by paragraph. Edit either side and the matching paragraph of the other file is updated in place, with the rest unchanged.

Writing assistant
Polish (as you ask, when you write a request), ask, write new paragraphs in the manner of a reference paper, write figures (LaTeX code, a PDF picture or an editable PowerPoint diagram, traced from an image or in the style of one), fix LaTeX errors, or rewrite a translation and let the paragraph follow. A reference can be added as a file or taken from the materials folder, packages a figure needs are added, and in a LaTeX paper *Accept* compiles before it applies. Jobs run in the background, and you decide what is applied.

My materials
The folders of research-builder's `materials/`. Your papers give the voice to write in, your papers and figures serve as references, and your icons go into PowerPoint figures.

Paper templates
VLDB, ICLR 2027 and IEEE TII built in. Your own projects can be imported as templates, and the open project exported.

APIs and key pools
OpenAI-compatible, OpenAI Responses and Anthropic APIs. A provider holds several keys, uses them evenly, moves to another key on rate limits, and takes a new key in place of an old one.

Request log and updates
The log records the model, reasoning effort and service tier of every request. Any version published on GitHub can be installed from inside the app.

Close to the tray
Closed, the window hides and BiWrite stays in the menu bar or notification area with the document and its translations. A click on the icon brings it back.

Figure 1 shows the main window: **(1)** is the source, **(2)** the translation.

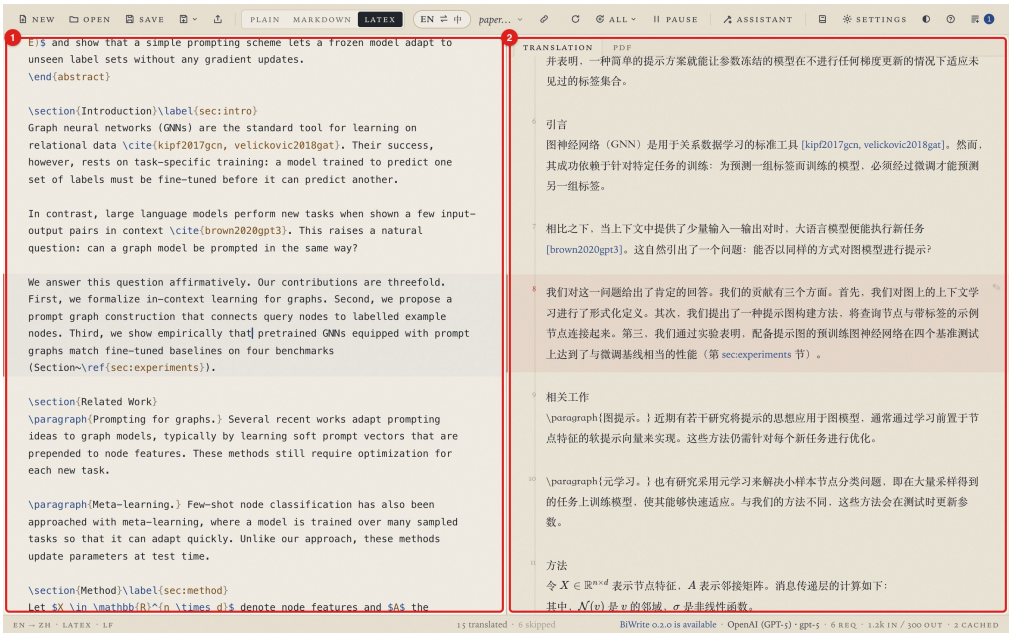


Figure 1 The main window: (1) English LaTeX on the left, (2) the Chinese paragraph by paragraph on the right

2 The writing workflow

This section follows one paper from a new project to submission in ten steps. Each step gives the essentials and points to the section with the details.

2.1 Step 1: A new paper from a template

Click *New* in the toolbar, find the venue in *Papers and templates* (Figure 2 **(1)**), click *New paper…* **(2)** and name the folder of the paper. BiWrite copies the template and opens its main file. Section 4.8 shows how to add templates of your own.



Figure 2 (1) The ICLR 2027 template, (2) New paper

2.2 Step 2: Writing with the translation alongside

Write English on the left. 0.8 seconds after you stop typing, your changes reach the translation queue, and a changed paragraph is translated once it has stayed unchanged for 5 more seconds. The right pane then shows its Chinese (Figure 1). Settings can change the 5 seconds to anything from 0 to 60 (Section 10.6). A first draft in Chinese works too. When BiWrite finds the text is Chinese, it translates it into English instead (Section 3.4).

2.3 Step 3: Writing in the manner of a published paper

For paragraphs with a familiar shape, such as an introduction or related work, the writing assistant can write in the manner of a published paper.

1. Put the cursor in the paragraph the new text should follow, press ⌘J and choose *Write* (Figure 3 **(1)**). **(2)** shows the paragraph the new text follows. With the cursor on an empty line, the new text goes there.
2. Write what you want at **(3)**: the subject, how many paragraphs, how long. For example, *two paragraphs motivating in-context learning on graphs, written like the introduction of the reference paper*.
3. Click + *File* **(4)** in the *References* row and choose the paper to imitate, such as the `.tex` file of the UniClean paper. A LaTeX file gives its body without comments, and a long paper is cut after 120,000 characters. The chip shows the file name and its length **(5)**. + *Text* adds a pasted passage instead.
4. Click *Run* **(6)**.

The model follows how the reference paper arranges its paragraphs, its sentence patterns and how it argues, and takes the content from your instruction and the paper around the cursor. It does not copy the reference's sentences, claims, numbers or citations. A result your paper does not give yet appears as a marked placeholder such as `[result]` for you to fill in.

The card (Figure 4) shows the new paragraphs **(1)**, their Chinese paragraph by paragraph **(2)** and notes on how they were written **(3)**. *Accept* **(4)** puts them after the target paragraph, and the right pane shows the Chinese from the card without another request (Figure 5).



Figure 3 (1) Write, (2) where the new text goes, (3) what to write, (4) add a reference paper, (5) the reference added, (6) Run

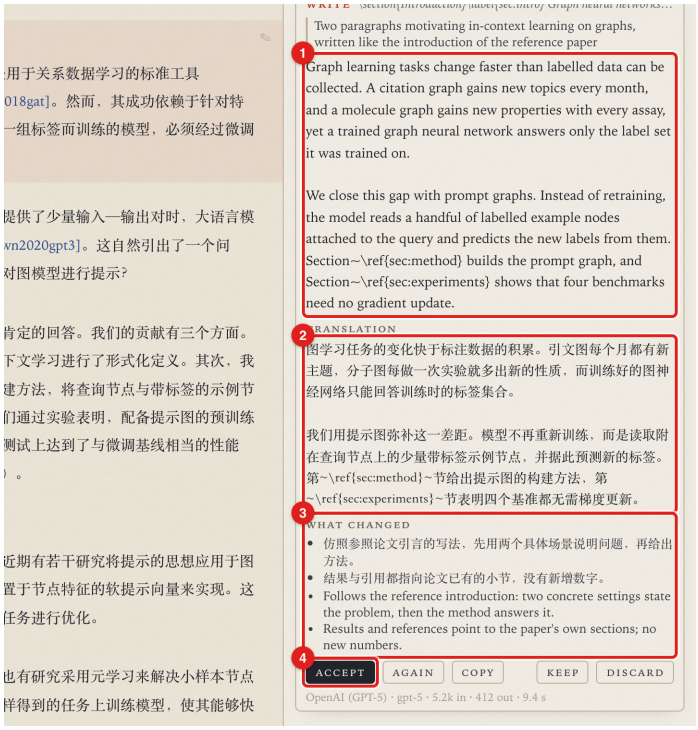


Figure 4 (1) The new paragraphs, (2) their Chinese, (3) notes, (4) Accept

2.4 Step 4: Polish, edit and ask

Polish evens out the wording, or changes a paragraph as you say when you write a request in the box, and *Ask* checks an argument. Section 6 has the details.

2.5 Step 5: Figures and tables

Choose *Figure* (Figure 6 **(1)**), describe the figure or table **(3)**, for example *accuracy on the four benchmarks as a table*, and click *Run* **(4)**. In a LaTeX paper, *Format* **(2)** chooses what it becomes.

LaTeX code
LaTeX ready to insert, as described below.

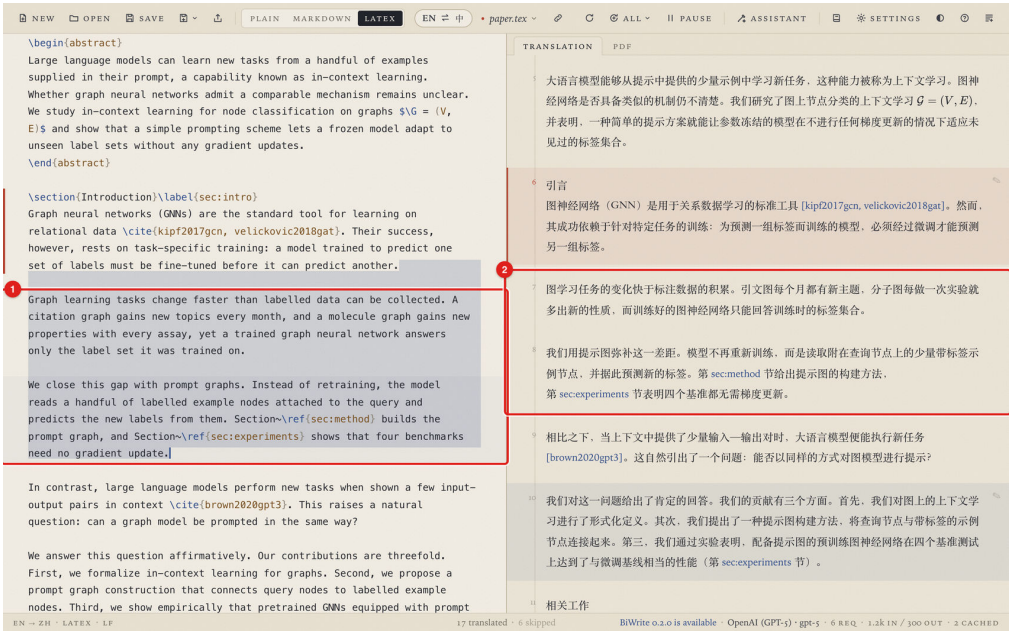


Figure 5 (1) The new paragraphs on the left, (2) their Chinese on the right

PDF picture

TikZ built into a PDF of its own and included with `\includegraphics`. When it does not compile, BiWrite has the model repair it first.

PowerPoint

An editable PowerPoint diagram of native shapes, block arrows and SVG icons, laid out by the figure rules of `paper-figure-pptx` in the writing skill. With LibreOffice installed, BiWrite also turns it into the PDF the paper includes. Without LibreOffice only the PowerPoint file is made, and the card says how to export the PDF from PowerPoint.



Figure 6 (1) Figure, (2) Format, (3) the request, (4) Run

The card of a PDF picture or a PowerPoint diagram (Figure 7) shows the picture **(1)**. **(2)** *Save as* saves a copy as PowerPoint, PDF or PNG where you choose. **(3)** names the files that saving the document writes, into the folder of the paper's other figures, such as `figures/`, under names not taken yet. Until you save, the files live only in the app and in the cache folder `.biwrite/` beside the paper, and a build of the unsaved document still finds them. **(4)** *Accept* puts the figure environment that includes it into the paper.



Figure 7 (1) The picture, (2) Save as, (3) the files written on save, (4) Accept

With *LaTeX code*, the card gives LaTeX ready to insert (Figure 8 **(1)**). The request lists the packages the document loads and asks for code that uses them first. When the code needs a package the document does not load, the card names it **(2)**. If the open file is the main file, *Accept* **(3)** puts the code after the paragraph and adds the `\usepackage` lines right before `\begin{document}` (Figure 9). In a section file, the card names the lines to add to the main file's preamble. Accepting it has the model fit the code and compiles the document first (Section 6), and code that still does not compile once applied is for *Fix*.



Figure 8 (1) The table, (2) the packages it needs, (3) Accept

A figure can start from reference images (Figure 10). + *Image* **(1)** in the *References* row adds an image file, and + *Materials* **(2)** takes a figure from the materials folder (Section 6.7). Each image is either *Trace* or *Style* **(3)**.

Trace

The image is a draft of this figure, such as a sketch or an older version. The model redraws it with its layout, parts, arrows and text, checks the text, symbols, formulas, numbers and steps against the paper, corrects what disagrees with the paper or is wrong in itself, and lists each correction in the notes. With *Context* set to *Whole document*, it checks against the whole paper.

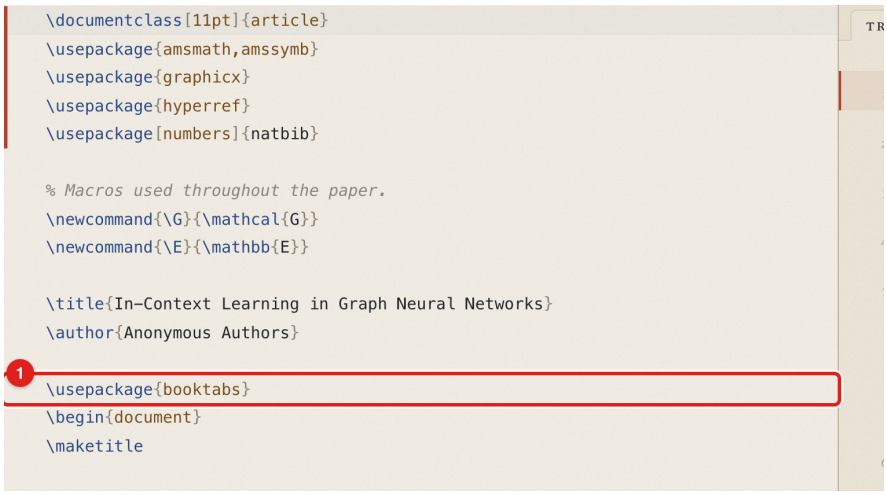


Figure 9 (1) The package added to the preamble on Accept

Style
The image is a figure of another paper. The model takes only its layout, colours, lines, arrows and type sizes, while the content comes from your request and the paper, never the text or numbers of the image.

Images can be PNG, JPEG, WebP or GIF, and SVG, PDF and PowerPoint figures of the materials folder are turned into images first. Each can be up to 8 MB, and the model must accept images.



Figure 10 (1) Add an image, (2) take a figure from the materials, (3) Trace or Style

2.6 Step 6: The PDF, and editing from it

Open the PDF tab and press ⌘B. A click on a sentence in the PDF selects it in the source, ready to polish or edit (Section 4). The line numbers in the margin of a submission do not get in the way.

2.7 Step 7: Editing the Chinese

Click *EN* ⇄ 中 to edit the Chinese on the left, with the English following (Section 3). To change one paragraph only, click the pencil on its Chinese on the right and rewrite the Chinese with *Via translation*. The English is revised to say the same (Section 5).

2.8 Step 8: The Chinese PDF and .tex

In the PDF tab, switch to *Chinese*, compile, and save the PDF and the Chinese `.tex` with *Save PDF as…* and *Export Chinese .tex…* (Section 4.6).

2.9 Step 9: Keeping the Chinese version in step

Put the exported Chinese `.tex` next to the English main file as `paper_zh.tex`. The next time the English file opens, the two pair up, and an edit on either side updates the matching paragraph of the other file in place (Section 5).

2.10 Step 10: Saving and keeping versions

⌘S saves, and *Save As* keeps the versions of each stage. Once the paper is done, *Export this project…* under *New* packs it as a template for the next paper.

3 Everyday writing

3.1 Open and edit

Open in the toolbar (Figure 11 **(1)**, ⌘O) opens a `.tex`, `.md` or `.txt` file. A LaTeX project of several files lists its files in the title bar **(2)**, to move between them. Each block on the right is one paragraph, heading or caption of the source. Click a block and the cursor moves to its paragraph, which is highlighted on both sides **(3)**. A click on a sentence of the translation selects the same sentence of the source, or only moves the cursor to the paragraph when the two have different numbers of sentences. With the cursor in the source, ⌘⌥J (Ctrl+Alt+J) marks its sentence in the translation for a few seconds. When the right pane shows the PDF, the same keys find the cursor in the PDF.

Only the paragraphs you changed are translated again. Each is sent once it has stayed unchanged for the wait set in Settings (5 seconds by default), and its translation streams in as it arrives. A language swap, the PDF of the other language, an export and saving a paired file send the waiting paragraphs at once. Math, citations, references, labels, URLs and comments reach the model as placeholders and come back exactly as they were.

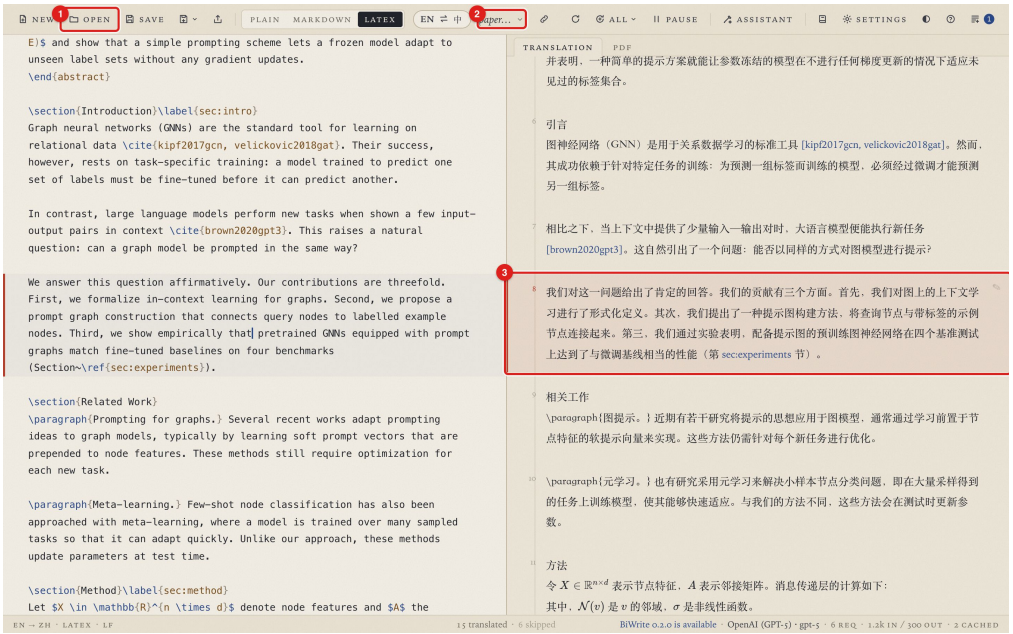


Figure 11 (1) Open a file, (2) the files of the project, (3) the paragraph's translation

3.2 Switching the edited language

The *EN* ⇄ 中 button in the middle of the toolbar (Figure 12 **(1)**) sets which language you edit on the left.

1. By default you edit English on the left and read Chinese on the right.
2. After the swap, the left pane holds the Chinese (Figure 13 **(2)**) and you edit it, with the English on the right **(3)**. Only the paragraphs you change get new English, revised from your previous English. Every other paragraph keeps your exact English.
3. Click again **(1)** to edit English. Paragraphs you did not change come back word for word.

A swap needs every paragraph translated. If some are still being translated, the button waits and swaps once they are done. A second click swaps at once. The paragraphs not translated yet keep their text for the moment, are translated in the background and are filled in on the left as their translations arrive (Figure 14). A paragraph you edit before that keeps your edit. When you swap back, their English comes back word for word. While translation is paused nothing is filled in, and *Continue with the rest* in the *All* menu does it.

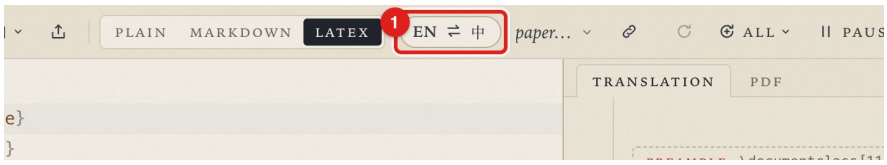


Figure 12 (1) The button that swaps the edited language

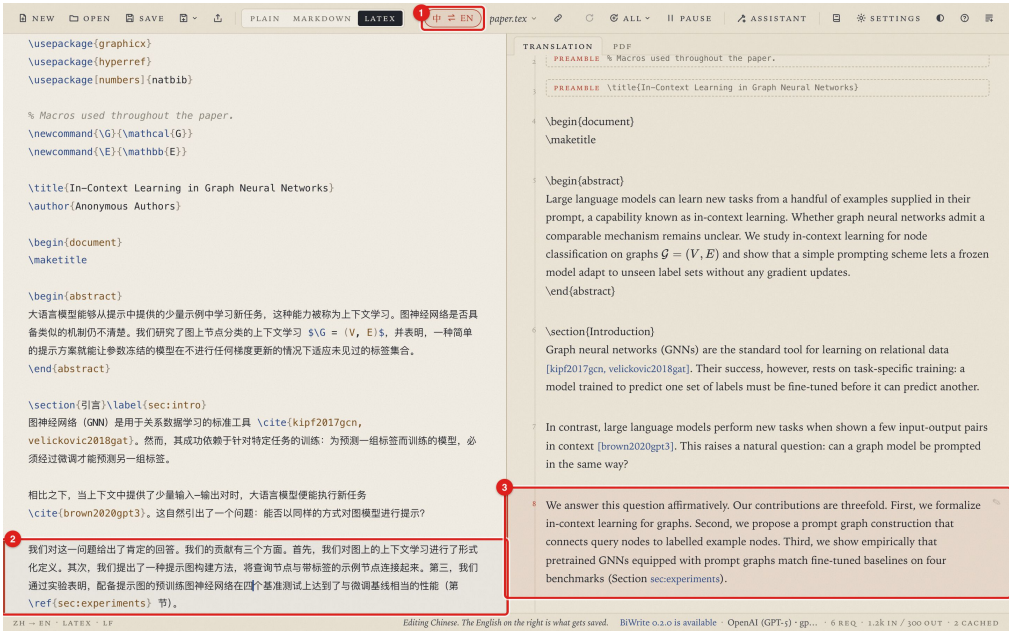


Figure 13 After the swap: (1) the button reads 中 ⇄ EN, (2) the Chinese is edited on the left, (3) the English follows on the right

Saving always writes the file's own language. While you edit the Chinese of an English file, saving writes the English composed from the translations. With a paired Chinese file, both files are written (Section 5).

3.3 The All menu

All in the toolbar (Figure 15 **(1)**) opens a menu.

- **(2)** *Retranslate every paragraph* sends every paragraph again, without the cache. It is off while you edit the other language, since the right pane then holds the text of your file.
- **(3)** *Continue with the rest* translates only paragraphs without a translation or whose translation failed, also while translation is paused. After an early swap, it also fills in paragraphs on the left still in the file's language.



Figure 14 Swapping early: (1) editing the Chinese, (2) a paragraph still being translated, (3) the note in the status bar

- **(4)** *Read the document as Chinese, translate into English* (in a Chinese paper, the opposite) is for a document whose language was taken wrongly (Section 3.4).



Figure 15 (1) The All menu, (2) retranslate every paragraph, (3) continue with the rest, (4) change the document's language

3.4 A paper written in Chinese

Opening a file, BiWrite judges its language from the paragraphs of the text, with math, citations and LaTeX commands left out. When Chinese clearly prevails, you edit the Chinese on the left with the English translation on the right, and the status bar reads zh □ en (Figure 16). Then

- saving writes the Chinese on the left. The swap button lets you edit the English instead, and saving writes the Chinese composed from the translations back into the file,
- *Chinese* in the PDF tab compiles the Chinese file on disk, *English* compiles the English translation, and the export button reads *Export English .tex*….

In a mixed document the judgment can be wrong, and you can set it right. When the text on the left plainly is the other language, the swap button reads it as such at once, without waiting (Figure 17). Otherwise choose *Read the document as Chinese, translate into English* or the opposite in the *All* menu. The file stays as it is and is translated again from its language.

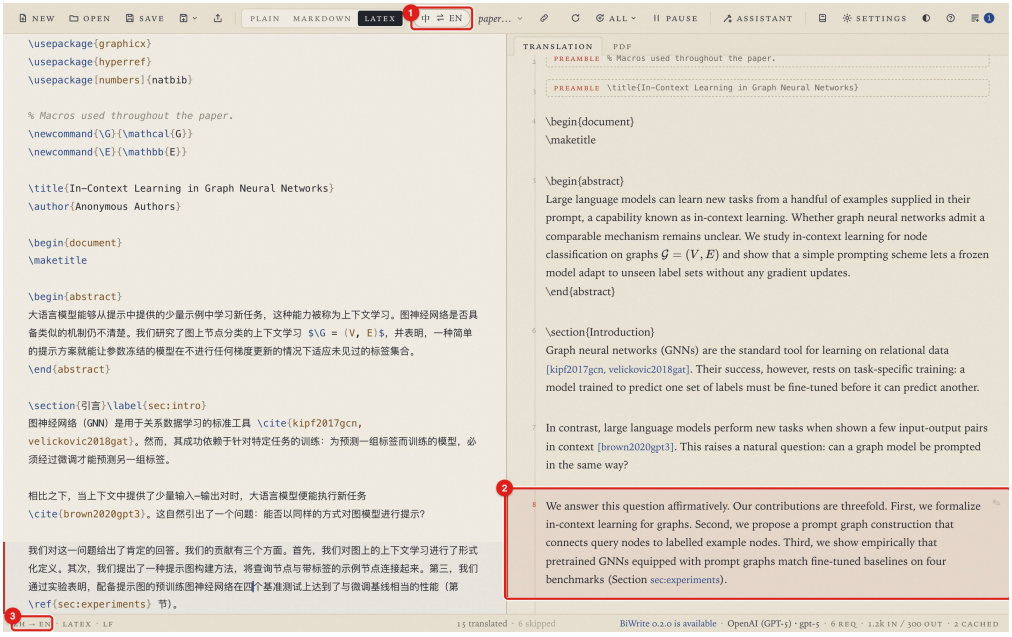


Figure 16 A Chinese file: (1) the button reads 中 ⇄ EN and is not swapped, (2) the English translation on the right, (3) zh □ en in the status bar

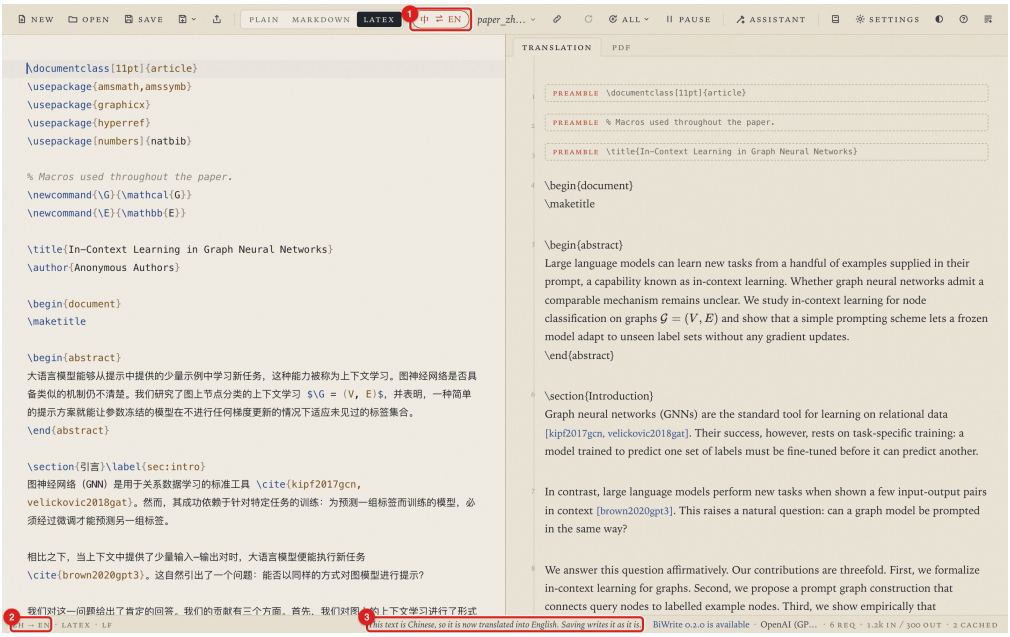


Figure 17 The swap button on a Chinese file taken for English: (1) the Chinese is now the source, (2) the status bar reads zh □ en, (3) the note

3.5 Save and Save As

Save (Figure 18 **(1)**, ⌘S) overwrites the file. *Save As* **(2)** (⌘⇧S) saves under a new name and suggests one not taken yet, for example `paper-2.tex`. The original stays as it was.

In a project of several files, the *Save As* menu also has *Save as one .tex (sections merged)…*. It puts the main file and every section it includes into one file, in order, a page break kept around each `\included` one, with the open file as edited, and the project stays as it is. When a section is saved under a new name that the main file does not include, BiWrite says that the main file's PDF leaves the new file out. To save the whole paper elsewhere, use *Save with the template into a new folder…*, which keeps the main file and the sections in their folders, and the sections paired with *Pair* stay paired in the new folder.

When another program changes the open file or its paired file on disk (another editor, git, a sync service), BiWrite notices when its window gains focus and asks whether to load the latest version. *Load* puts it in. With unsaved changes the button reads *Load (discard unsaved changes)*. A paired file is paired again, and the document keeps your edits. *Ignore* changes nothing, and that version is not asked about again. Saving never overwrites a change made elsewhere without asking: it offers *Overwrite*, *Save As…* and *Cancel*. A paired file changed elsewhere is not written until you choose *Overwrite*. Each of these notices has *Show the changes* (Figure 19 **(1)**), which shows line by line how the file on disk differs from the version read when you began editing (Figure 20). It is there to look at and leaves the choice to you. After Save As, when the original file was changed elsewhere while you edited, a notice says so and the original stays as it is.

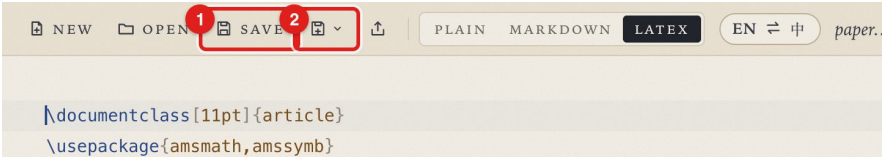


Figure 18 (1) Save, (2) Save As

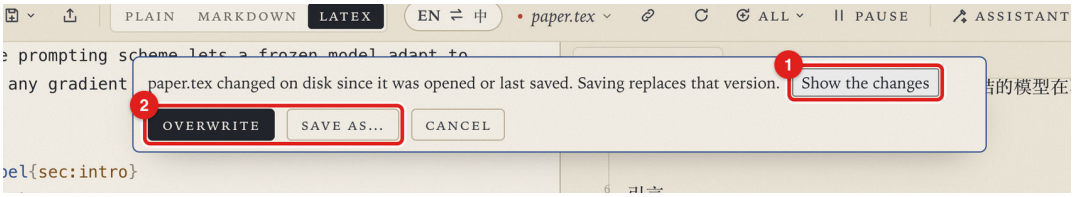


Figure 19 Saving a file changed elsewhere while you edited: (1) Show the changes, (2) Overwrite or Save As

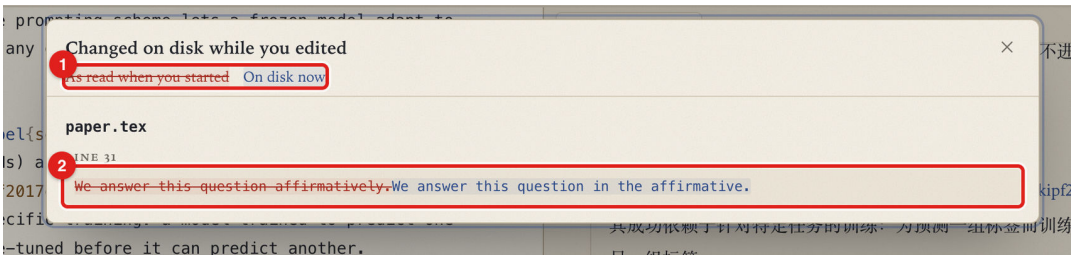


Figure 20 (1) What each side is, (2) the changes line by line, the removed struck out and the added coloured

3.6 Glossary

Glossary in the toolbar opens the glossary (Figure 21). In the table **(1)**, set your preferred Chinese for a term or tick *Keep EN*. The buttons **(2)** import and export CSV files with the columns `term,translation` (UTF-8). After a change, only paragraphs that mention the term are translated again.

4 LaTeX papers

4.1 Open a project

Open a `.tex` file with *Open*, or click *New* (Figure 22 **(1)**) and choose *Open LaTeX folder…* to open a project by its main file. A project of several files lists them in the title bar **(2)**, with the main file marked. The right pane of a LaTeX document has two tabs, *Translation* and *PDF* **(3)**. A project file that `\input` or `\include` reads has an arrow after its name in the editor. A click on it, or a ⌘-click on the name, opens that file. With a section open, *Main file* in the title bar goes back to the main



Figure 21 The glossary: (1) terms and their Chinese, (2) import, export and save

file. In a project of several files, the *Translation* tab switches between *This file* and *Whole paper*. The whole paper follows the main file's includes to any depth. Other files' paragraphs come from their paired Chinese files or the translations in the cache, a paragraph without one shows its own words in a faint colour, and looking requests nothing. A click on one opens its file with that paragraph selected.

BiWrite finds the main file from a `% !TEX root = …` comment, else from the `\input` and `\include` chains that reach the open file, however deeply nested. The engine comes from a `% !TEX program` comment, else the project's `latexmkrc` (`$pdf_mode`), else the packages of the preamble.



Figure 22 (1) New and Open LaTeX folder, (2) the files of the project, (3) the Translation and PDF tabs

4.2 Compile the English PDF

1. Click the *PDF* tab (Figure 23 **(1)**).
2. Click *Compile* **(2)** or press ⌘B (Ctrl+B on Windows). It compiles what the editor shows, with no need to save first (see the next section).
3. The PDF appears. **(3)** shows the time taken and the numbers of errors and warnings, and the problem list **(4)** links each one to its line.

Compile after saving a .tex file is on by default, so every save compiles again. A document with errors still gets its PDF, with the errors listed. *Fold* at the top right of the list folds it away, and a click on the build result **(3)** shows it again. *Copy log* copies every problem and the build output. When a picture the document names is not on disk yet and the document gives its web address (a download command in a comment, say, ending in the file's name), the PDF pane lists it, and *Download into the paper's folder* fetches it and builds again. Only pictures and PDFs are fetched, only into the paper's folder, and never over a file that is there. A project with its own build settings, `latexmkrc`, is noted once, on its first build. Such a file can run any command, so look at it first in a project from someone else.

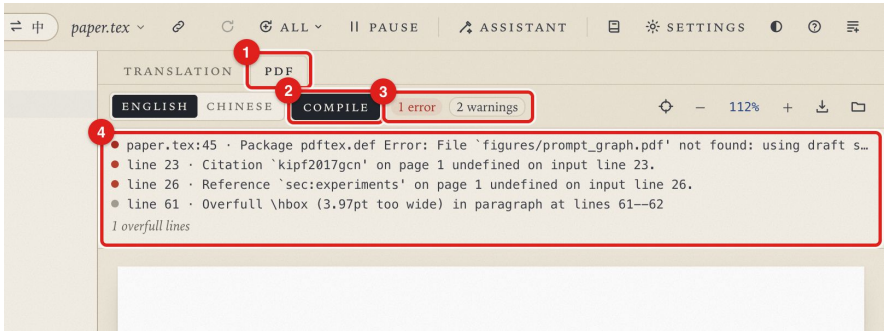


Figure 23 (1) The PDF tab, (2) Compile, (3) the result, (4) the problem list

4.3 Compile without saving

A build of a document with unsaved edits leaves the files on disk as they are. BiWrite writes what the editor shows into the hidden folder `.biwrite/` beside the paper and builds the PDF there, and the `.tex` file on disk stays as last saved until you click *Save* or *Save As*. When the file you edit is a section of a larger project, a copy of the main file includes the edited section, the other files are used as they are, and the bibliography is found as well. PDF pictures and PowerPoint diagrams of the writing assistant also wait in that folder until you save, and builds include them. Every task of the writing assistant works on what the editor shows too, saved or not.

In a few cases the edited section cannot go into the copy, for example when a custom macro includes it. The PDF is then built from the files as saved, and BiWrite says that the edits appear once you save. A new document that was never saved needs to be saved as a `.tex` file before it compiles.

4.4 From the PDF to the source

Click a sentence in the PDF (Figure 24 **(1)**) and the same sentence is selected in the source **(2)**. A small menu next to the click **(3)** offers *Polish*, *Polish…* and *Ask…* on that sentence. *Polish* polishes by the writing rules at once, and *Polish…* opens the writing assistant to write a request first. If the sentence is in another file of the project (such as `sections/04_method.tex`), BiWrite opens that file and selects it there, after asking when the open file has unsaved changes.

BiWrite checks the words under the click. Things drawn over the text, such as the line numbers in the margin of a submission or a running header, mislead SyncTeX, and then BiWrite looks the clicked words up in the open file and in the other files of the project.

4.5 From the source to the PDF

Put the cursor in a paragraph (Figure 25 **(1)**) and click the locate button above the PDF **(2)**, or press ⌘⌥J (Ctrl+Alt+J). The paragraph is highlighted in the PDF for a few seconds **(3)**.

4.6 Compile the Chinese PDF after translation

The Chinese PDF is the translation set as a document, compiled with XeLaTeX with Chinese font support (the ctex package) added.

1. Open the English paper and wait until every paragraph is translated. The status bar counts translated, in-flight and pending paragraphs.
2. In the PDF tab, click *Chinese* (Figure 26 **(1)**). The first switch compiles it, and *Compile* **(2)** builds it again later.
3. Paragraphs not translated yet are set in English, and a note above the PDF counts them **(3)**.

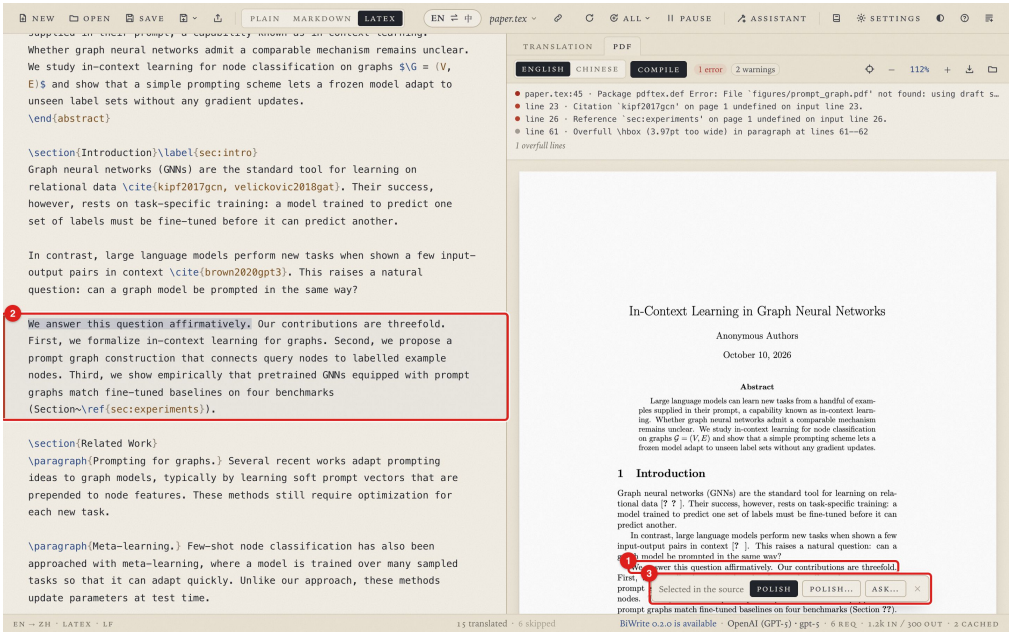


Figure 24 (1) A click on a sentence in the PDF, (2) the sentence selected in the source, (3) the menu

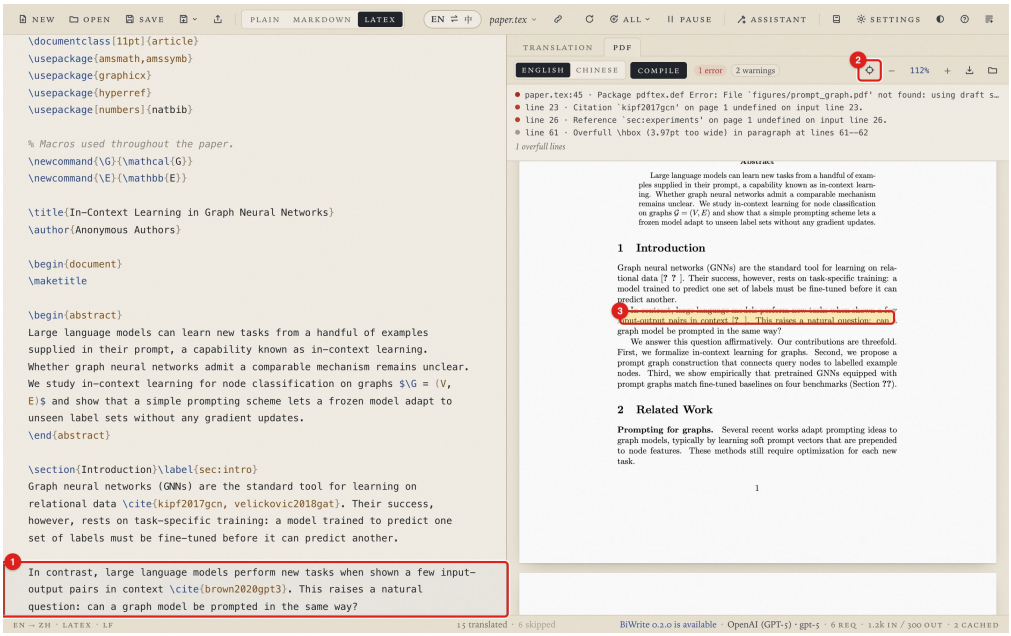


Figure 25 (1) The paragraph at the cursor, (2) the locate button, (3) its place in the PDF

The Chinese PDF and its build files are kept in the hidden folder `.biwrite/zh/` of the paper's folder. None of your files are touched.

In a project of several files, the Chinese PDF is the whole paper whichever file is open. A Chinese main file of the project's own (`main_zh.tex`, or one paired with the main file by *Pair*) that includes the open section, or its Chinese file, is built as it is. Otherwise the main file's title and abstract and each other section come from their paired Chinese files, else from the translations in the cache, and no translation is requested for them. A click in the Chinese PDF opens the English section at that paragraph.

4.7 Export the Chinese PDF and .tex

On the Chinese PDF (Figure 27):

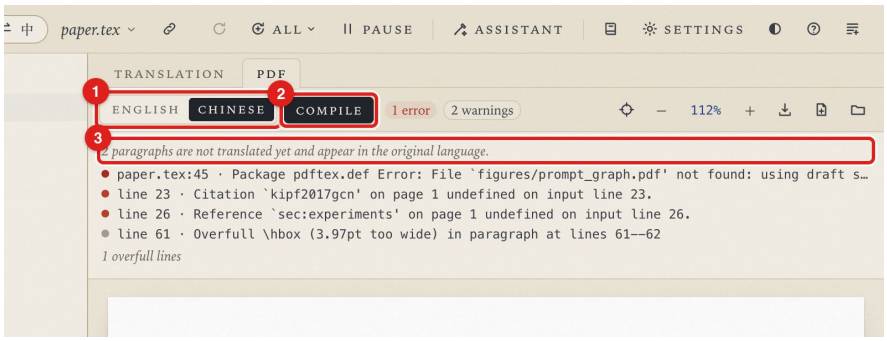


Figure 26 (1) Show the Chinese PDF, (2) Compile, (3) paragraphs not translated yet

- **(1)** *Save PDF as…* saves a copy of the Chinese PDF, named `paper.zh.pdf` by default. The same button saves the English PDF.
- **(2)** *Export Chinese .tex…* saves a LaTeX file that compiles on its own with XeLaTeX, named `paper_zh.tex` by default. A Chinese paper exports its English version here instead, with *Export English .tex…* on the English PDF. In a project of several files the export is the whole paper with the translations of every section, named after the main file, to compile beside it.
- **(3)** *Show in folder* shows the PDF in Finder or Explorer.



Figure 27 (1) Save PDF as, (2) Export Chinese .tex, (3) Show in folder

4.8 Paper templates

New opens *Papers and templates* (Figure 28).

- The built-in templates are VLDB (PVLDB volume 20, 2027), ICLR 2027 and IEEE TII. The first two use the official template files, and TII comes with a short skeleton paper.
- **(1)** *New paper…* copies a template into a folder you name and opens the main file. The PDF tab compiles it.
- **(2)** *Open LaTeX folder…* opens an existing paper project.
- **(3)** *Import folder…* and *Import .zip…* keep an existing project as a template of your own, listed under *My templates*.
- **(4)** *Export this project…* zips the open project with a template manifest, for others to import.

5 An existing Chinese version

If you keep a Chinese version by hand (`paper_zh.tex` next to `paper.tex`), BiWrite uses it instead of translating. Once the two files are paired, editing the English updates the matching paragraph



Figure 28 (1) A new paper from a template, (2) Open LaTeX folder, (3) import a template, (4) export the project

of the Chinese file, and editing the Chinese updates the English file.

5.1 Automatic pairing

Opening an English file looks for its Chinese counterpart by name and pairs them when their paragraphs match:

- a suffix in the same folder: `paper.tex` with `paper_zh.tex`, or `.zh`, `-zh`, `_cn`,
- parallel language folders: `sections_en/04_method.tex` with `sections_zh/04_method.tex`, `en/` with `zh/`.

5.2 Pair by hand

1. Open and save the English file.
2. Click *Pair* (Figure 29 **(1)**, the link icon) next to the file name in the title bar.
3. Choose the Chinese file. BiWrite aligns the two files paragraph by paragraph. The title bar shows the Chinese file (Figure 30 **(1)**), the status bar counts the paired paragraphs **(2)**, and the right pane shows your Chinese **(3)**.

Paragraphs pair by kind (paragraph, heading, caption), shared citation keys, references, labels, math, numbers and names, and length. A figure placed elsewhere in the Chinese file still finds its partner. Paragraphs without a partner are translated, but their translation is not written into the Chinese file.



Figure 29 (1) Pair

5.3 Editing and saving a pair

- The right pane shows your Chinese, with no requests.
- Edit an English paragraph, and only its Chinese is revised, from your Chinese.

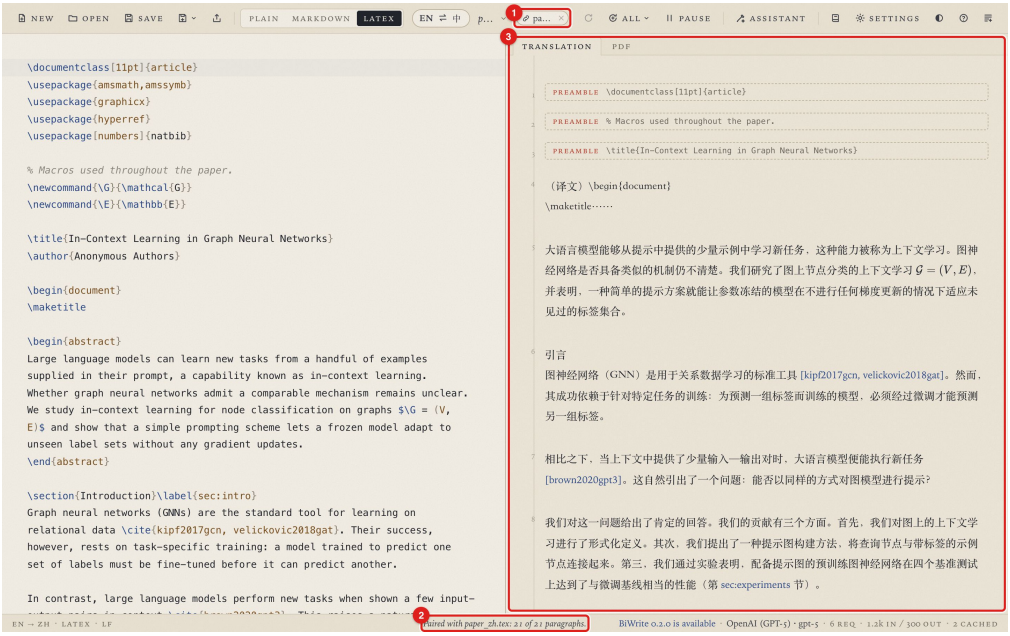


Figure 30 After pairing: (1) the Chinese file, (2) the paired paragraphs, (3) your Chinese on the right

- You can also edit the Chinese directly. The pencil on a paragraph on the right (Figure 31 **(1)**) opens *Via translation* in the writing assistant with the paragraph's Chinese in the box **(2)**. Rewrite it and click *Run* **(3)**: the model revises the English as little as possible to say the same. The card shows the change to the English (Figure 32 **(1)**), and *Accept* **(2)** applies it, with your sentence as the paragraph's Chinese.
- Saving writes both files. Only changed paragraphs are replaced in the Chinese file, and the rest stays byte for byte. If a translation is still on its way, the English is saved at once and the Chinese file follows when the translation arrives.
- After the swap, you edit your Chinese file on the left, and changes go back into the English file the same way.
- *Save As* (Figure 33 **(1)**) saves both files under the new name, for example `paper-2.tex` and `paper_zh-2.tex`. The original two stay as they were.



Figure 31 (1) The pencil, (2) the paragraph's Chinese to rewrite, (3) Run



Figure 32 (1) The change to the English, (2) Accept

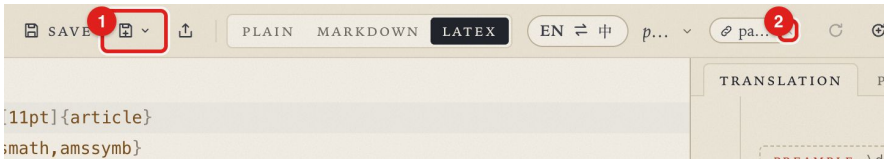


Figure 33 (1) Save As, (2) unpair

5.4 PDFs of a pair, and unpairing

With a pair, *Chinese* in the PDF tab compiles your own Chinese main file (`paper_zh.tex`), not a document assembled from the translation. A click in the Chinese PDF selects the matching English paragraph. The cross on the Chinese file in the title bar (Figure 33 **(2)**) ends the pair, and saving no longer writes the Chinese file.

6 Writing assistant

6.1 Open it and pick one of six tasks

Assistant in the toolbar or ⌘J opens the writing assistant on the right (Figure 34). It works on the selection, or on the paragraph at the cursor **(2)**, as the editor shows it, saved or not.

1. Pick a task at **(1)**.

Polish
Applies the writing rules. It removes AI-style phrasing and dashes or semicolons that splice sentences, and keeps claims, numbers, citations and formulas. With a request in the box, for example *shorter, state the number of benchmarks*, it changes the text as asked, by the same rules. ⌘⇧P polishes the paragraph at the cursor.

Ask
Answers a question about the selection or the whole paper, and leaves the text as it is.

Write
Writes new paragraphs as you ask, inserted after the paragraph, in the manner of a reference paper when you add one (Section 2, step 3).

Figure
Writes LaTeX code, a PDF picture or a PowerPoint diagram from a description, inserted

after the paragraph with the packages it needs, traced from a reference image or in its style (Section 2.5).

Via translation
Fills the box with the paragraph's translation. Rewrite it and run, and the paragraph is revised to say the same (Section 5).

Fix
Compiles the whole document, gives TeX's errors to the model and offers the changes that make it compile (Section 6.4). It appears in LaTeX documents only.

2. Write the instruction or question at **(3)** (polishing needs none).
3. **(4)** *Context* sends the text alone, with its neighbouring paragraphs, or with the whole paper. In a project of several files the whole paper is put together from the main file, with the open file as edited.
4. **(5)** *References* takes passages whose style to follow (+ *Text*), whole papers as `.tex`, `.md` or `.txt` files (+ *File*), images (+ *Image*), or papers and figures of the materials folder (+ *Materials*, Section 6.7).
5. Click *Run* **(6)**. The job runs in the background while you keep writing.

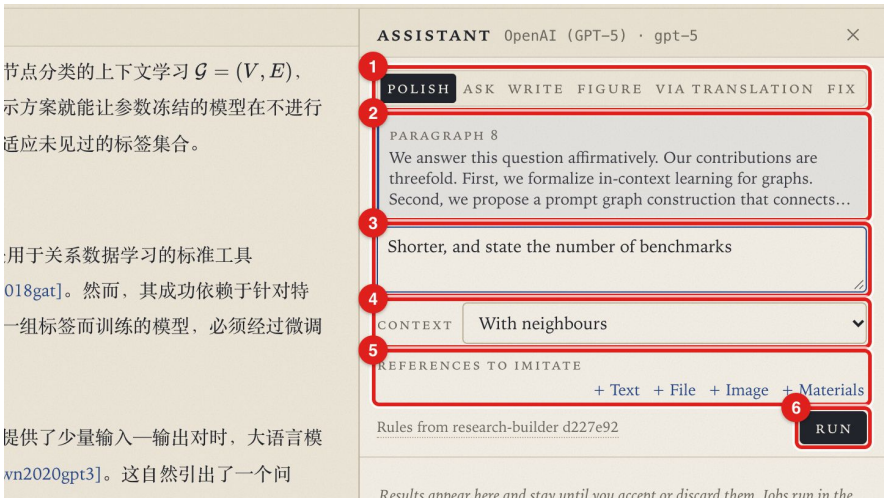


Figure 34 (1) Task, (2) target, (3) instruction, (4) context, (5) references, (6) Run

6.2 Decide on the result

The finished card (Figure 35) shows the changes as a diff **(1)** (deleted words struck out, new ones highlighted), the translation of the revision **(2)** and the reasons for the changes in Chinese and English **(3)**. *Accept* **(4)** writes the revision where the text is now, even after you kept typing. If the target itself was edited meanwhile, the card says so, and *Re-apply with the model* brings the revision onto the current text. *Again* runs the job once more, *Keep* folds the card into one line to decide later (it can still be accepted when shown again), *Discard* drops it, and ⌘Z undoes an accepted revision. After Accept or Discard the card folds into one line too, and *Show* opens the result again. *Revert* on an accepted card undoes that one change and leaves later edits as they are, and the card can then be accepted again. Reverting an inserted figure takes it out at once, and the model then decides which of the lines it added before `\begin{document}` are still needed, keeping booktabs while another table has `\toprule`, say. The lines it picks are taken out of a copy that is built, and they go only when that brings no new error. Otherwise every line stays. The card names the lines taken out and those kept. When the model cannot be reached, only the figure goes and the lines stay. *Stop* on the card ends that check and leaves the lines as they are. If the text it put in was edited since, nothing

is overwritten and the card shows the text from before, to copy and put back by hand. *Delete* takes the card off the list.

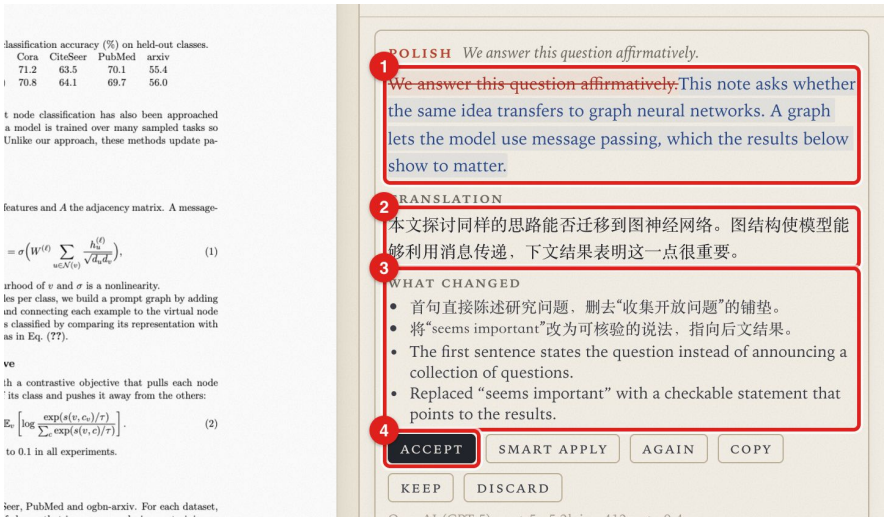


Figure 35 (1) The diff, (2) the translation, (3) the reasons, (4) Accept

6.3 Accepting with the model's fit and a build

In a LaTeX paper saved as a `.tex` file, *Accept* (Figure 36 **(1)**) does not apply the result at once. Its code may use a package, a TikZ library or a colour the document lacks, and the document would then fail to compile. BiWrite first gives the result and the document to the model to fit it to the document, using the packages and colours the document has and adding the lines it needs before `\begin{document}`, and the card shows *Fitting it to the document*. The document is then compiled with the result, which is applied when that brings no new error. When it brings new errors, the model repairs them, in at most three rounds, and the result is applied once it compiles. Errors the document had before do not count. The applied card says the model fitted it or repaired it to compile (Figure 37 **(1)**), and shown again it lists the changes made to compile **(2)** and the lines added to the preamble **(3)**. When errors are left after three rounds, or neither the model nor a build can be used, the model is left out and the result is applied as it is, the card says why, and *Fix* can follow. *Stop* on the card cancels the check, and nothing is applied. While one card is accepted or reverted, *Accept* and *Revert* of the others wait for it, and the editor takes no typing until it is done. In a new document not saved yet, in Markdown and in plain text, *Accept* applies the result at once. *Accept and Revert with the model's help*, in the *Writing assistant* section of Settings (Figure 51 **(2)**), is ticked by default. Unticked, Accept applies the result at once and Revert takes out only the text. All of this changes only what the editor shows, builds are made in the cache folder `.biwrite/`, and only *Save* or *Save As* writes your files.

In a project of several files, each card belongs to the file and the language it was made in. With another file open, or the other language, the card names its file, and *Switch there* goes back to accept it. A card whose text changed meanwhile so that its place is gone says so and cannot be accepted. While a card is accepted or reverted, opening another file or swapping languages waits for it. When a figure inserted in a section needs packages the main file's preamble lacks, the check adds them to its copy of the main file, and the card names the lines to add there.

6.4 Fix

When the document does not compile, choose *Fix* (Figure 38 **(1)**) and click *Run*. The box can stay empty or name the problem to fix. BiWrite compiles the whole document and gives TeX's errors with their lines to the model, the model proposes changes, and BiWrite compiles the changed text

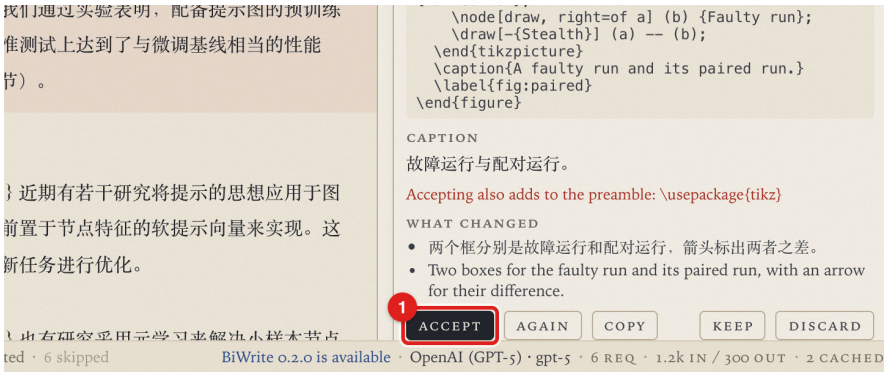


Figure 36 (1) Accept



Figure 37 (1) Repaired to compile, (2) the changes made to compile, (3) the lines added to the preamble

again, for at most three rounds. The card says how many errors there were and whether the changes make the document compile **(2)**, and lists each change by its line **(3)**. *Accept* **(4)** applies them, and *Revert* takes them back. The changes go into the editor only, and no file is saved. With a section open, only that file's errors go to the model. The errors of the project's other files are listed apart on the card. A click on a file opens it at the error, and *Fix* can then run there.

6.5 Questions

An answer appears in the card (Figure 39 **(1)**), with a box for a follow-up question **(2)**. A question card has no *Accept* or *Discard*. *Delete* takes it off the list, and *Keep* folds it to read later. Writing new paragraphs and figures are shown in Section 2, steps 3 and 5.

6.6 The writing skill

The rules come from research-builder, an open-source writing skill under the MIT license, each task with its part of the skill. Polishing and edits use the rules on AI-style phrasing and the writing playbook, *Write* adds the paper anatomy and the storytelling patterns, figures use the figure style, and



Figure 38 (1) Fix, (2) the summary, (3) the changes by line, (4) Accept

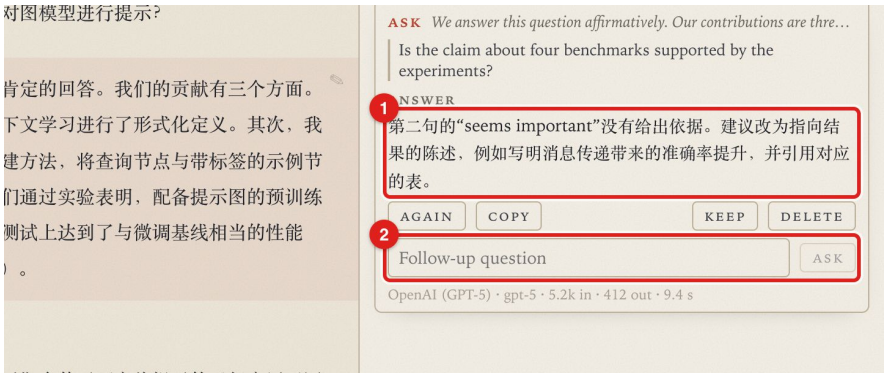


Figure 39 (1) The answer, (2) a follow-up question

PowerPoint diagrams use the rules and icon library of `paper-figure-pptx`. BiWrite ships a copy. The *Writing skill* section of Settings (Figure 40) can **(1)** update it from GitHub, **(2)** use a skill folder of your own, and **(3)** open the skill's repository. With a skill folder of your own, its `materials/` is the materials folder.

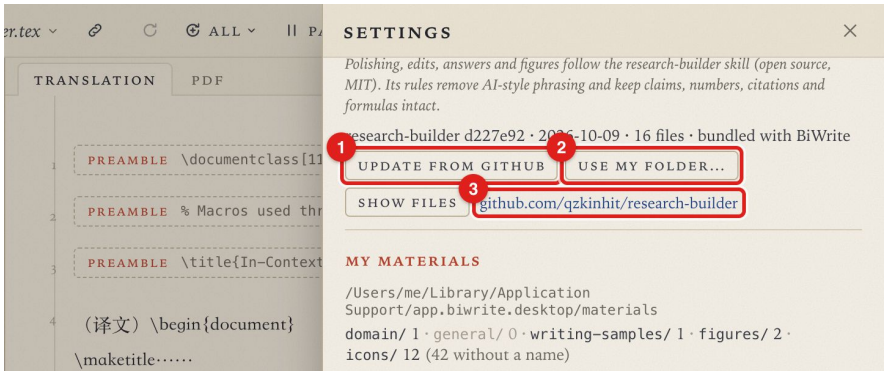


Figure 40 (1) Update from GitHub, (2) use your own folder, (3) the skill's repository

6.7 My materials

The published skill holds none of the author's papers, figures or icons, so BiWrite keeps a materials folder with the same folders as research-builder's `materials/`. Put each file into its folder. With a skill folder of your own, the materials folder is its `materials/`. Otherwise it lives in BiWrite's data folder, apart from the skill, so updating or changing the skill keeps it. The *My materials* section of Settings (Figure 41) shows where the folder is **(1)** and how many usable files each folder holds **(2)**, and *Show folder* **(3)** opens it in the Finder or Explorer. Files put into the folder show when you come back to the BiWrite window.

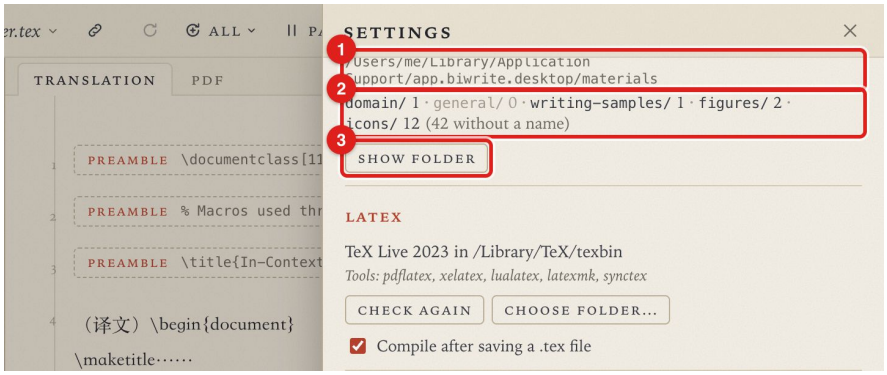


Figure 41 (1) The materials folder, (2) the files of each folder, (3) Show folder

Folder	What goes there	How BiWrite uses it
`writing-samples/`	Passages or papers in your own voice, as `.tex`, Word, `.md` or `.txt`	The openings of the first three are samples of the voice for polishing, edits, *Via translation* and *Write*
`domain/`	Your papers, as LaTeX sources (a folder each), Word, PDF, `.md` or `.txt`	References under + *Materials*. A LaTeX paper of several files is read through its main file
`general/`	Grant proposals and reports	The same
`figures/`	Figures you like, as PNG, JPEG, SVG, PDF or PPTX	Reference images under + *Materials*, to trace or take the style of. PPTX needs LibreOffice
`icons/`	SVG icons, or PPTX files holding SVG icons	Icons of PowerPoint diagrams. An SVG file is named by its file name, an icon in a PPTX by the picture's alternative text, and one without alternative text is left out (Settings shows how many)
`ppt/`	Talks and posters	Not used

7 Request log

The log button at the right of the toolbar (⌘⇧L) opens the request log (Figure 42). **(1)** filters the records, pauses recording, clears the log and opens the log file. Above the table, each purpose shows its count, and a click shows only its requests (another click shows them all). Each row of the table **(2)** is one request, with the model, reasoning effort and service tier that were sent and the ones the server declared, marked where they differ. A click on a row shows its details (Figure 43 **(1)**): the HTTP status, time to the first token, duration, input and output tokens (cached and reasoning tokens included), the key used (its name and last characters), and notes on retries and failover,

followed by the system prompt, the message sent and the reply, each kept up to 20,000 characters. The log stays on this computer and never holds keys.

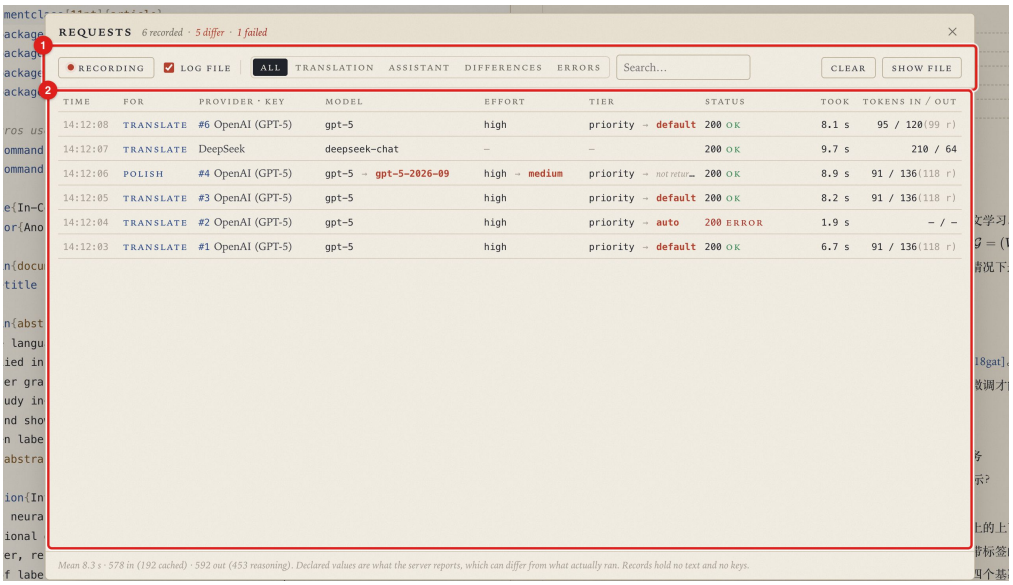


Figure 42 (1) Filters and actions, (2) the requests

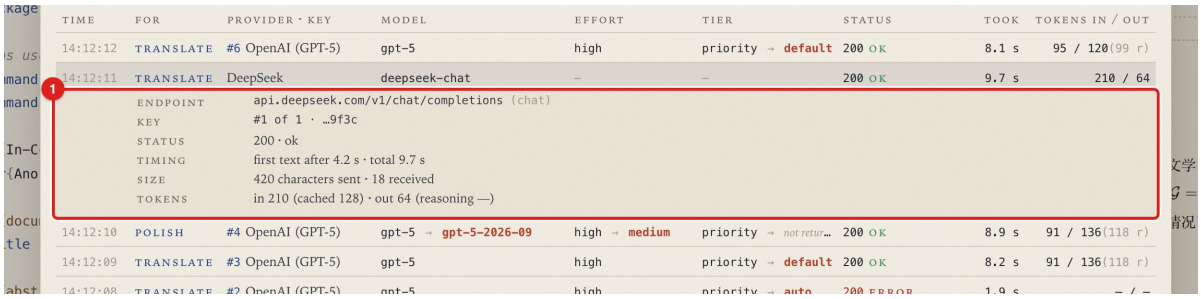


Figure 43 (1) The details of one request

8 Updates

In Settings, under *Updates*, *Show versions* (Figure 44 **(1)**) lists every release on GitHub with its notes **(2)**, older versions and pre-releases (branch builds) included. Pick a version and click *Install* **(3)**. BiWrite downloads the installer and checks it against the SHA-256 that GitHub reports. On macOS, *Restart now* (Figure 45 **(1)**) starts the new version. On Windows, the installer runs after BiWrite quits. At startup, BiWrite looks for a newer release and says so in the status bar, which can be turned off.

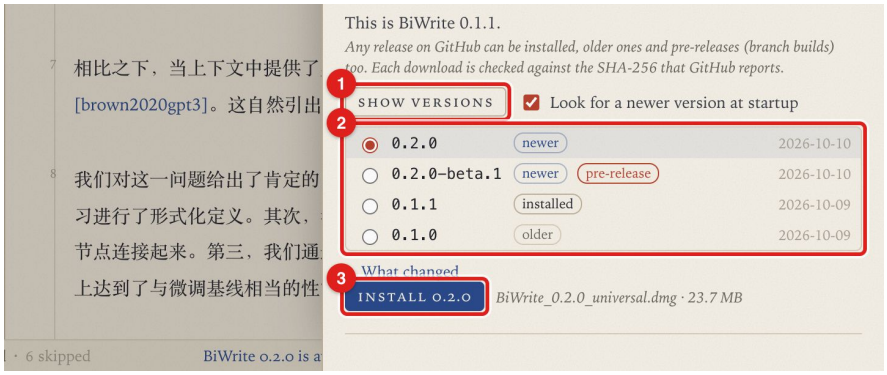


Figure 44 (1) Show versions, (2) the releases, (3) Install



Figure 45 (1) Restart after the installation

9 Install

9.1 macOS

Download `BiWrite_<version>_universal.dmg` from the Releases page (`https://github.com/zzccppp/biwrite/releases`). It runs on Apple silicon and Intel Macs with macOS 12 or later. Open the disk image and drag BiWrite into *Applications*.

The app is not signed with an Apple developer certificate, so macOS blocks it the first time. To allow it:

1. Double-click BiWrite. macOS says it cannot verify the developer. Click *Done*.
2. Open *System Settings* and go to *Privacy & Security*.
3. Under *Security* near the bottom, macOS shows *"BiWrite" was blocked to protect your Mac*. Click *Open Anyway*.
4. Confirm with *Open Anyway* again and enter your password or use Touch ID.

After this, BiWrite opens normally. In Terminal, this command has the same effect:

```
xattr -dr com.apple.quarantine /Applications/BiWrite.app
```

9.2 Windows

Download `BiWrite_<version>_x64-setup.exe` for 64-bit Windows 10 or 11. The installer has no trusted code signature either. When SmartScreen shows *Windows protected your PC*, click *More info*, then *Run anyway*, and follow the installer.

9.3 A TeX distribution

LaTeX papers need a TeX distribution: MacTeX on macOS, TeX Live or MiKTeX on Windows. BiWrite looks for it in the usual places and shows what it found in the *LaTeX* section of Settings (Figure 46 **(1)**). For a distribution somewhere else, *Choose folder…* **(2)** picks the folder that holds `pdflatex`. **(3)** turns compiling after every save on or off.

10 Set up a model provider

10.1 Supported APIs

Translation and the writing assistant use a language model API. BiWrite supports:

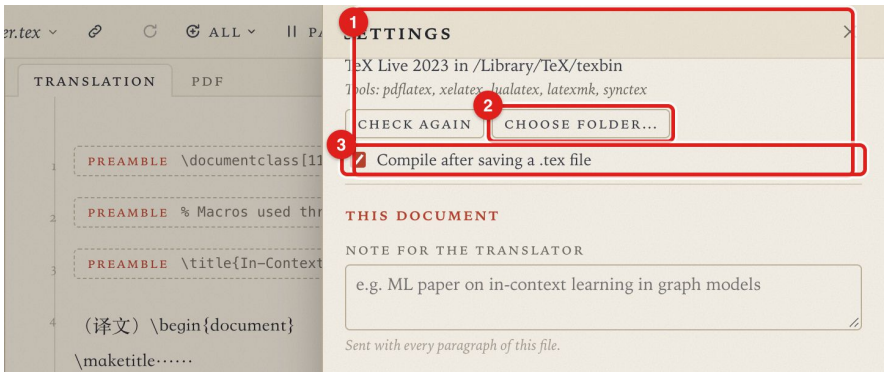


Figure 46 (1) The TeX distribution found, (2) Choose folder, (3) compile after saving

- OpenAI-compatible Chat Completions APIs: OpenAI, DeepSeek, Qwen, Kimi, OpenRouter and local servers such as Ollama or LM Studio,
- the OpenAI Responses API, for reasoning models that take a reasoning effort and a service tier,
- the Anthropic Messages API.

The built-in *Mock* provider works offline and returns each paragraph reversed, for trying the interface without a key.

10.2 Add a provider

1. Open *Settings* (⌘,). **(1)** lists the providers, and the radio button selects the one that translates (Figure 47).
2. Choose + *Add provider…* at **(2)** and pick a preset.
3. In the provider's form (Figure 48), check the base URL, the model and the API type **(1)**, and choose the reasoning effort and service tier **(2)** where the API takes them.
4. Paste the key and click *Add provider*. Keys are kept in a file of BiWrite's settings folder that only your account can read, are never uploaded, and are never shown again. Starting or updating BiWrite asks for no keychain password. Unticking *Keep API keys on this computer, without the keychain* in Settings moves them to the system keychain (the Credential Manager on Windows). *Manage keys…* **(3)** manages them later.
5. *Test* **(4)** asks the model a small question drawn at random and shows the question with its answer. *Test* on a key of the pool does the same.

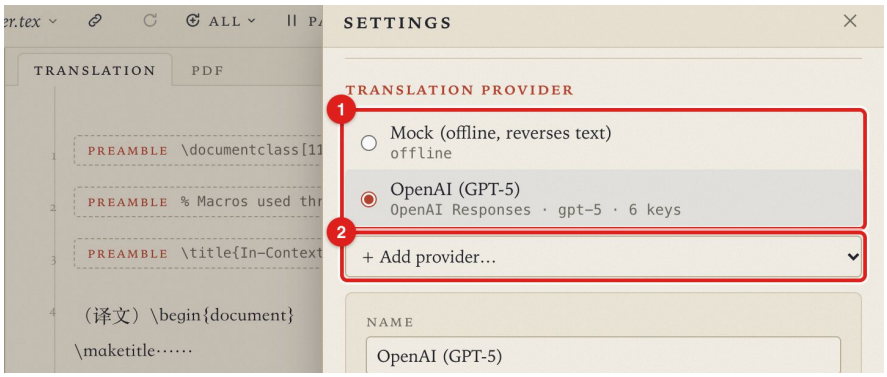


Figure 47 (1) The providers, (2) add a provider



Figure 48 The provider's form: (1) API type and temperature, (2) reasoning effort and service tier, (3) Manage keys, (4) Test

10.3 Third-party APIs

A third-party service is set up like any other provider. Its preset sets the API type, the address, the model, the reasoning effort, the service tier, the requests per key and the retries. Paste one or more keys and save. A service without a preset takes the address, model and API type from the provider's documentation. The request log shows the reasoning effort and service tier the server reports back, so you can check that a third-party service honours them.

10.4 Key pools

A provider can hold several keys, a pool. *Manage keys…* in the provider's form opens it (Figure 49).

- **(1)** lists the keys. Give each a name. The table shows only the last characters of a key, its state and its requests in flight. Each request goes to the least busy ready key. A key hitting a rate limit cools down and the request moves to another key, and a rejected key is set aside.
- **(2)** sets the requests per key, as many as the provider allows, and the retries.
- **(3)** takes new keys: one per line, or a name on one line and its key on the next, or *name: key*. *Add keys* adds them to the pool.
- **(4)** *Import from file…* and *Export to file…*: the export saves the names and keys to a text file to move them to another computer or share them with a colleague, and the import adds the keys of such a file. The keys go to the file without passing through the window. The file holds the keys in plain text, so share it only with people you trust.

New key on a key's row (Figure 50 **(1)**) replaces that key with a new one, under the same name and in the same place of the pool. Paste the new key into the box **(2)** and click *Save* **(3)**. It takes one key at a time, and not one the pool already has. Use it after creating a new key for the same account in the provider's console, to replace an old key that stopped working or has never been answered. *Ignore* beside it sets the key aside: it takes no request, shows as *ignored*, and the parallel requests count the other keys only. *Use again* takes it back.

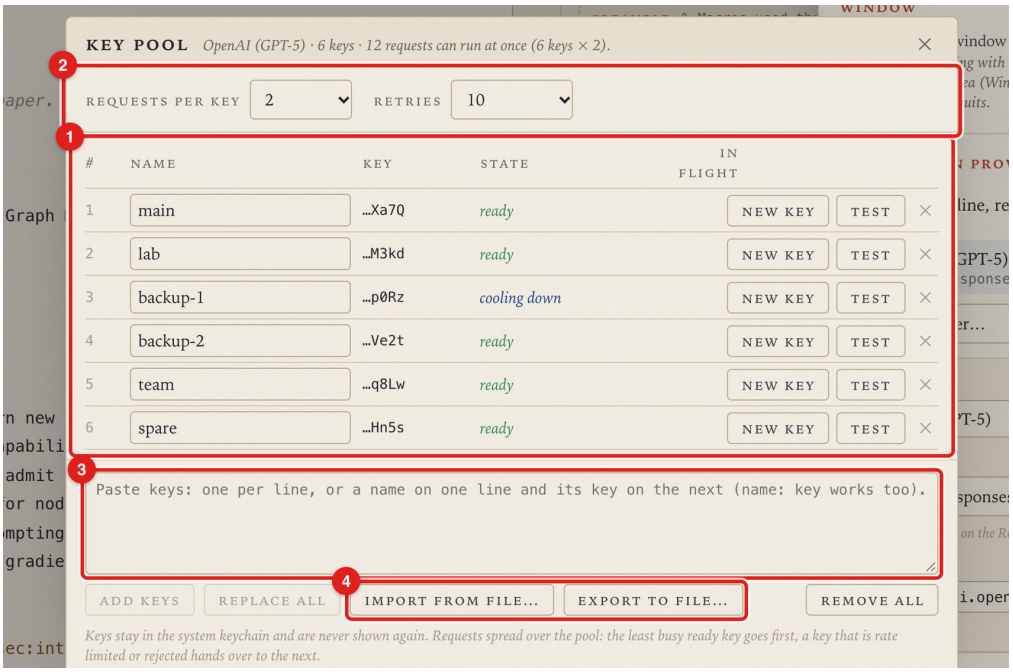


Figure 49 The key pool: (1) the keys, (2) requests per key and retries, (3) new keys, (4) import from and export to a file



Figure 50 (1) New key, (2) the new key, (3) Save

10.5 Separate models for translation and writing

The *Writing assistant* section of Settings (Figure 51 **(1)**) picks the model for polishing, edits and questions. A cheap, fast model can translate while a stronger one writes. *Same as translation* uses one provider for both.

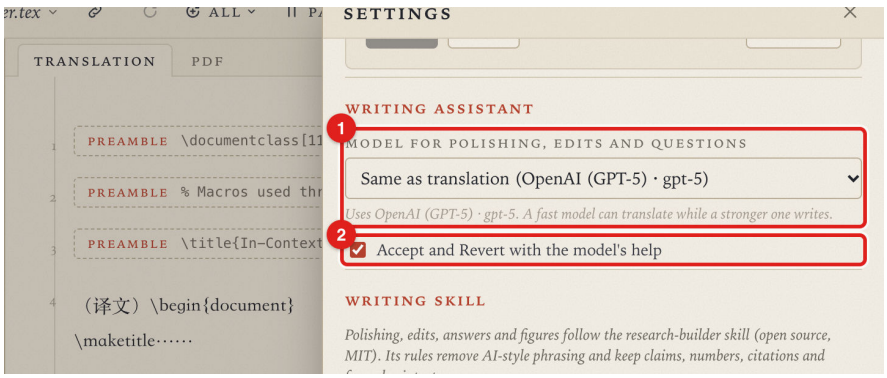


Figure 51 (1) The model of the writing assistant, (2) Accept and Revert with the model's help

10.6 Request pacing

The *Requests* section (Figure 52) controls how many requests reach the provider.

- **(1)** Wait after an edit: 5 seconds by default, 0 to 60. An edited paragraph is sent for translation once it has stayed unchanged this long, so editing it does not send a request at every pause. At 0 it is sent at once.
- **(2)** Paragraphs per request: 1 by default. With 2 to 8, several new paragraphs share one request, which suits providers with rate limits. Edited paragraphs are always sent alone, so their translation is revised minimally.
- **(3)** Parallel requests: the number of requests at once.
- **(4)** With *Use the whole key pool*, it is the number of keys times the requests per key.

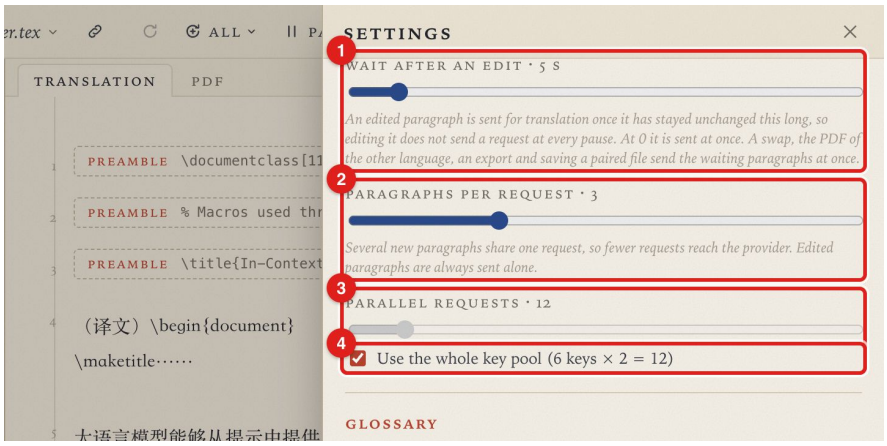


Figure 52 (1) Wait after an edit, (2) paragraphs per request, (3) parallel requests, (4) follow the key pool

10.7 Interface language

Language at the top of Settings (Figure 53 **(1)**) switches the interface between English and Chinese. Translation directions do not change.



Figure 53 (1) The interface language

10.8 Closing the window and the tray

In the *Window* section of Settings (Figure 54 **(1)**), *Closing the window hides BiWrite in the tray* is on by default. Closed, the window hides and BiWrite keeps running with the open document, and translations and assistant jobs go on. The BiWrite icon in the menu bar (macOS) or the notification area (Windows) offers *Show BiWrite* and *Quit BiWrite*, and on macOS the Dock icon brings the

window back too. Quitting with unsaved changes asks first. Turned off, closing the window quits BiWrite.

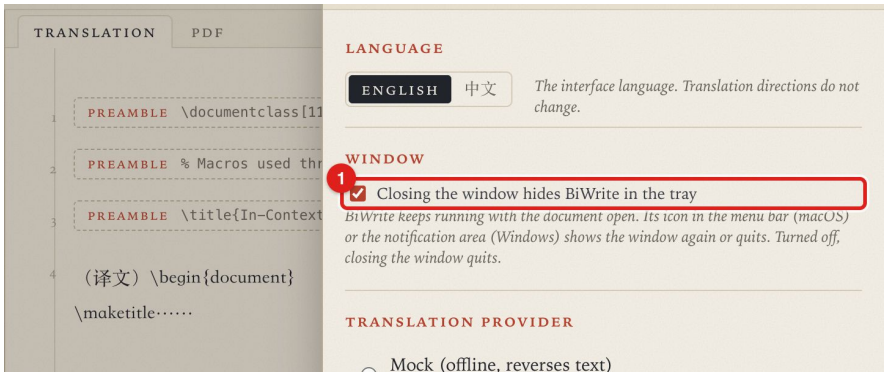


Figure 54 (1) Closing the window hides BiWrite in the tray

11 Keyboard shortcuts

macOS (Ctrl for ⌘and Alt for ⌥on Windows)	Action
⌘O	Open a file
⌘S / ⌘⇧S	Save / Save As
⌘B	Compile the PDF on show
⌘⌥J	Find the cursor in the translation or the PDF
⌘J	Open or close the writing assistant
⌘⇧P	Polish the paragraph at the cursor
⌘K	Focus the assistant's input
⌘,	Settings
⌘⇧L	Request log
⌘F	Find and replace in the source
⌘Z	Undo (accepted revisions too)

12 Questions

No TeX distribution was found. Install MacTeX, TeX Live or MiKTeX and click *Check again* in the PDF tab. For a distribution in another place, choose its folder in the *LaTeX* section of Settings.

The build failed or the PDF is old. Read the problem list above the PDF and click an error to reach its line. For a failure without a log, *Console output* shows the tools' output. A build stops after five minutes.

A click in the PDF does nothing. Every PDF that BiWrite builds has SyncTeX data. A click on an image or empty space shows that it has no source line.

A click says the PDF is being built again. After pairing or unpairing a translation, or swapping languages without a pair, the PDF built before no longer matches the text on the left, and the PDF tab says it is from before the change. A click or the cursor then builds that PDF again first, without saving any file. Click once more when it is ready.

The swap button keeps waiting. Some paragraphs are still being translated or failed. Click *Retry* on a failed paragraph or choose *Continue with the rest* in the *All* menu, or click the swap button again to swap at once.

A Chinese file shows Chinese on the right. BiWrite took the text for English, which happens with mixed documents. Choose *Read the document as Chinese, translate into English* in the *All* menu. The file stays as it is.

An inserted figure does not compile. Accepting it compiles and repairs it before it is applied. When it still does not compile, run *Fix* in the writing assistant. In a section file, check the packages named on the card and load them in the main file's preamble.

A PowerPoint diagram has no PDF. Turning a PowerPoint diagram into a PDF needs LibreOffice. Install it and make the figure again, or open the PowerPoint file in PowerPoint and export a PDF of the same name into the same folder.

Icons of the materials folder are not used. An SVG file is named by its file name, so name it by what it shows, such as `data-pool.svg`. An icon in a PPTX needs its alternative text, set in PowerPoint. The *My materials* section of Settings shows how many icons lack one.

Where did BiWrite go after closing the window? It hides in the tray and keeps running. The BiWrite icon in the menu bar or notification area brings the window back, and the *Window* section of Settings turns this off (Section 10.8).

A third-party API often hits rate limits. Add several keys to the pool, set *Requests per key* to what the provider allows, raise *Paragraphs per request* and *Retries*. The request log shows each rate limit and each switch to another key.

The Chinese file does not pair. Files pair only when at least half of their paragraphs match. Check that the two files are the same document in two languages.

Where is the data kept? Translations are cached in `cache.sqlite3` in the app data folder. The *Translation cache* section of Settings shows its size and clears it. Keys are kept in `keys.json` in the settings folder, readable only by your account, or in the system keychain when the setting is unticked.